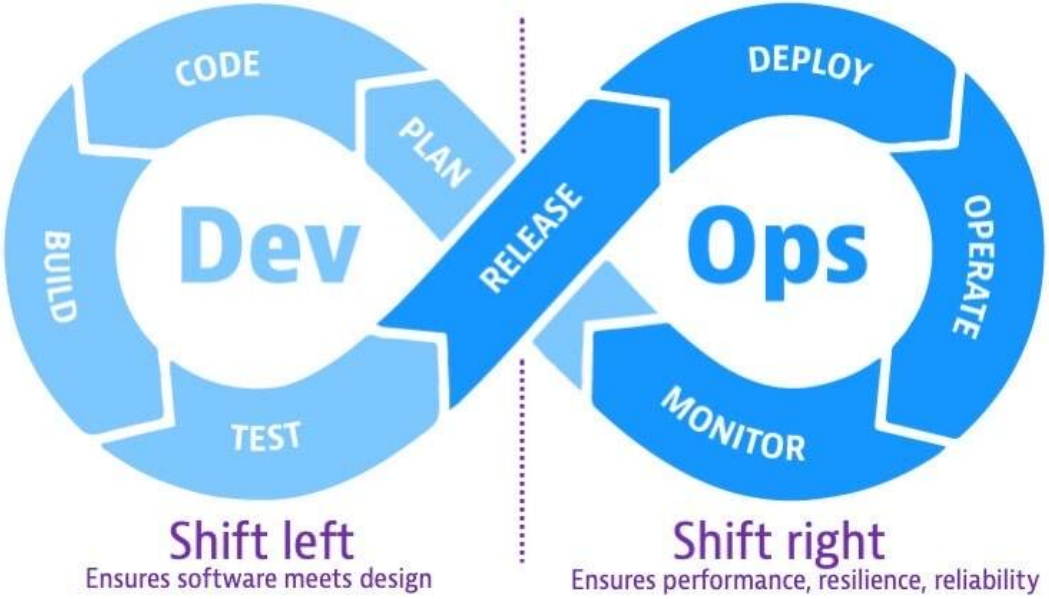
DevSecOps

Notes on DevSecOps
with a focus on open source softwares

What is this ?

- DevSecOps aims at integrating security in ALL phases of software development and deployment

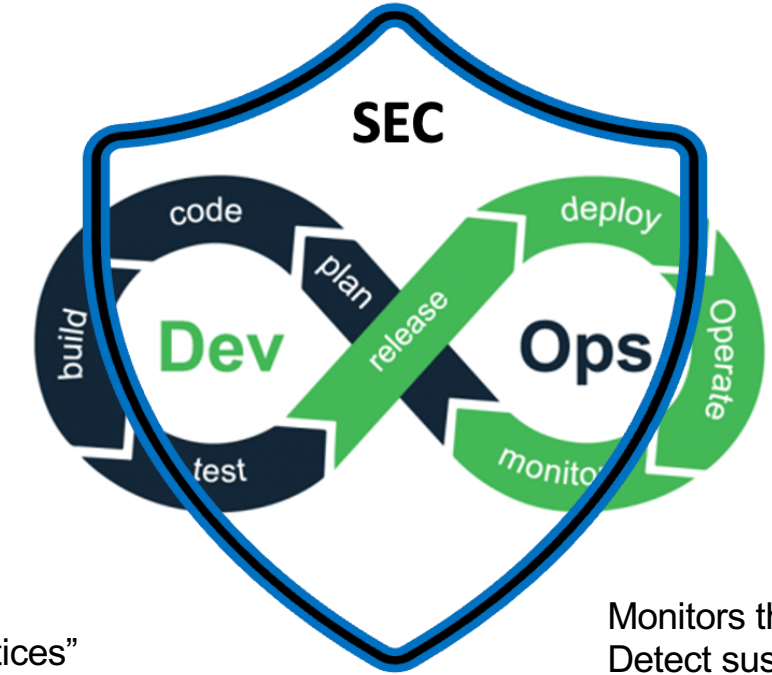
- DevOps automates software deployment. DevSecOps integrates the security controls at each stage, in an automated way.



(checks before the app is “online”)

(Deploy, then monitors the app while “online”)

DevSecOps: Focus on Security



Security by design
Enforce “good practices”
Identify known vulnerabilities
BEFORE the App is online

Monitors the app while “online”
Detect suspicious activities, potential attacks

Different angles for security enforcement

- Code
  - Detection of vulnerabilities
- Libraries, dependencies (& container layers)
  - SBOM (Software Bill of Materials) : identification of components
  - Detection of vulnerabilities in all these components
- APIs
  - Detection of vulnerabilities within APIs
- Posture (IAC):
  - CSPM (Cloud Security Posture Management). Detection of threats which are not due to the applications themselves but to its deployment in the cloud
- Antivirus
  - E.g. when uploads takes place
- … (not exhaustive)

Shift-Left

- **Static Application Security Testing (SAST)** - Analysis of an application's source, bytecode, or binary code for detecting security vulnerabilities.
- **Software Composition Analysis (SCA) and Vulnerability scanning**
  - Ex: Container image scanning (in cloud native framework)
  - Ex: VM image scanning
- **Secret leaking**
- **Code signing** - method of putting a digital signature on a program, file, or software update so that authenticity and integrity can be verified upon installation and execution.

om https://www.redhat.com/en/topics/devops/shift-left-vs-shift-right

Pre-deployment & Constant monitoring

- API **S**ecurity **T**esting
- **Dynamic application security testing (DAST) - a testing methodology used to uncover potential security flaws by performing automated security scanning against a running target.**
- **Chaos engineering tools: simulated failures to test the resiliency (& security)**

- Application Performance Monitoring (APM) tools: metrics, logs, and traces relating to application performance.
- SIEM (Security information and event management)
- IDS (Intrusion Detection)

To the left hand side: 1. Securing the App,

- **SAST**
  - SonarQube: https://github.com/SonarSource/sonarqube
  - Semgrep: https://github.com/semgrep/semgrep
  - Bearer: https://github.com/Bearer/bearer
  - Horusec: https://github.com/ZupIT/horusec
  - Code QL (integrated in github)
  - Bandit (python), Bearer(javascript,…), Mate (c/c++), blackslash, …

- **Secret detection**
  - Gitleaks
  - TruffleHog
  - …

TELECOM Paris
IP PARIS

To the left hand side: 2. checking libs, containers and VMs

- **SCA & Image Vulnerability Scanner**
  - **Trivy: https://trivy.dev/ (includes also posture management)**
  - **Grype: https://github.com/anchore/grype (vulnerability for containers)**
  - **Syft: https://github.com/anchore/syft (SBOM for containers)**
  - **VMClarity: https://github.com/openclarity/vmclarity (for VMs)**

See also: https://owasp.org/www-community/Free_for_Open_Source_Application_Security_Tools

To the left hand side: 3. Cloud Security Posture Management

- **CSPM**
  - Trivy (IaC, terraform, K8s)
  - Kubescore, Kubescape (K8s)
  - …

TELECOM Paris
IP PARIS

Pre-deployment:

- **DAST**
  - **Zed Attack Proxy (web app). https://www.zaproxy.org**
  - **Nikto**
  - **Many others…**
  - **(See https://github.com/paulveillard/cybersecurity-dast?tab=readme-ov-file)**

Constant surveillance:

- **Telemetry**
  - Collectors:
    - Prometheus, Cilium, Tetragon, … or offered as a service by cloud providers
    - (Usually based on eBPF for latest collectors)
    - Closed (paid) solution: Datadog, …
  - Standard format (OpenTelemetry)
    - For time series (network io, CPU, RAM),
    - logs,
    - sys_calls
    - traces (if service mesh used)
  - Pushed in databases (Elastic, …)
- **All-in-one packed solutions exists (e.g. DataDog, Splunk)**
- **Can be then be used for SIEM, IDS, …**
  - Wazuh
  - Many proprietary solutions

GIN303

Goal, objectives, practical guideline

GIN303

- **Project based**
  - **Learn by deploying, testing. Hands-on (experimental) approach**
- **Your project**
  - **Choose a set of tools (see previous slides) and test them.**
    - Loose requirements as difficulty depends on the set of tools tested etc.
  - **Goal: Improve security in general**
    - App/dev (git pipeline including SAST, BOM analysis, …)
    - DevOps: security of deployment
    - Infrastructure (Kind base deployment is far from meeting standard good practices)
  - **Based on your own interests and know-how, can cover all these aspects or, on the contrary, focus more on one particular aspect (app/dev, devops, infra)**
  - **Focus will be on K8s even if other aspects can also be studied (contact your professor)**

In practice…

- **Based on GIN302 VM**
  - **K8s Cluster:**
    - Kind already installed.
    - Alternative: As a team, start a new cluster by setting K3s (K8s for edge, i.e. small clusters) on each VMs. One “master” (control-plane+node), the other as nodes. OKD (community edition of RedHat Openshift) might also be an option (nice additions such as GUI, …)
  - **App:**
    - Ginflix. Code available at https://gitlab.telecom-paris.fr/jean-louis.rougier/ginflixv2
      - Includes code for front-end, streamer, back-end containers.
      - Does not include K8s deployment (based on your work in GIN302).
      - Can be used for Shift-left (SAST, BOM analysis, …) , in particular for setting a complete git pipeline.
    - Other open source examples exist eventually (if needed, contact me before using):
      - Google On Line Boutique (https://github.com/GoogleCloudPlatform/microservices-demo)
      - OWASP Juice Shop (but monolithic application, not “cloud native”)
      - …

Practical Guideline

- **Project definition**
  - **Initial project definition (tools, …) to be defined quickly**
  - **Contact me to validate it**
  - **(Of course it will most likely evolve…)**
- **Team work strongly advised**
  - **2-3 people (3 max)**
- **Don’t stay blocked (contact me).**
- **Regular project progress statement required**
  - **Starting in January: We can meet in 4B07 every Wednesday morning (I’ll be there from 9h30 to 12h00).**
  - **Each project should contact me at least once every two weeks (in particular if you can’t be there on Wednesday morning).**

Practical Guideline: Mandatory outcomes

- **Project defense will be organized in Feburary 2026**
  - **In front of a jury (colleagues, PhD students, external guests from Industry)**
  - **20 min presentation**
    - Prepare slides
    - Everybody should speak
    - A demo of at least one aspect would be a plus be nice (record something eventually if difficult to do real time)
- **Prepare a private git project (and invite me)**
  - **Containing all your work (code, etc.)**
  - **A readme file**
    - Documentation of your work.
    - Reproducing your work should be easy to do in 3 (or 6h) in a practical lab based on your guideline (GIN303 might become a “traditional” course with practical labs).